

Enhanced Report System - Complete Summary

What's New

Major Enhancements

1. Scenario-Based Filtering

- Replaced "Location" with "Scenario" selection
- Support for multiple scenarios (Accra, Kumasi, etc.)
- Independent data per scenario

2. Emergency Vehicle Analytics

- Real-time emergency vehicle tracking
- Response type breakdown (Ambulance, Fire, Police)
- Green wave activation statistics
- Priority-based analysis
- Scheduled vs detected comparison

3. Traffic Pattern Integration

- Display detected traffic patterns
- Confidence score indicators
- Pattern type categorization
- Traffic light-specific patterns
- Actionable recommendations

4. Actual File Generation

- PDF with professional formatting
- Excel with multiple sheets
- CSV raw data export
- Proper file downloads

5. Modern UI Redesign

- Gradient card designs
- Enhanced date range picker
- Loading animations
- Toast notifications

- Responsive layouts

Files Created/Modified

New Files

1. report_routes.py (Complete rewrite)

- 10 API endpoints
- PDF/Excel/CSV generation
- Emergency analytics
- Traffic pattern analysis

2. report.html (Complete redesign)

- New filter layout
- Emergency vehicle chart
- Traffic patterns section
- Enhanced UI components

3. requirements_report.txt

- reportlab==4.0.7
- XlsxWriter==3.1.9

4. Integration guides

- Step-by-step setup
- Troubleshooting
- Configuration examples

Database Models Used

Existing Models

-  TrafficLightLog
-  AIDecisionLog
-  TrafficPattern

Emergency Models (From your emergency_veh.py)

-  EmergencyVehicleLog
-  EmergencySchedule
-  GreenWaveSchedule

API Endpoints

Traffic Data

1. `GET /api/reports/traffic-overview`
 - Overall metrics with trends
 - Includes emergency count
2. `GET /api/reports/traffic-volume-trends`
 - Time-series vehicle data
 - Breakdown by type
3. `GET /api/reports/congestion-analysis`
 - Heatmap data (day x hour)
4. `GET /api/reports/vehicle-distribution`
 - Pie chart data

Emergency Analytics

5. `GET /api/reports/emergency-activity`
 - Emergency response trends
 - By type over time
6. `GET /api/reports/emergency-summary`
 - Priority breakdown
 - Type distribution
 - Scheduled vs detected

Traffic Patterns

7. `GET /api/reports/traffic-patterns`
 - Detected patterns
 - Confidence scores
 - Recommendations
8. `GET /api/reports/pattern-insights`
 - Pattern statistics
 - High-confidence patterns

AI Performance

9. `GET /api/reports/ai-performance`

- Decision metrics
- Optimization rates

10. GET /api/reports/traffic-light-performance

- Individual TL metrics
- Performance grading

Report Generation

11. POST /api/reports/generate

- Creates actual PDF/Excel/CSV
- Returns downloadable file

12. POST /api/reports/export

- Quick CSV export

UI Components

Filter Controls

html

- Scenario dropdown (replaces Location)
- Start date picker
- End date picker
- Time period selector
- Action buttons (Refresh, Export, Print)

Overview Cards (4)

1. Total Vehicle Count (with trend)
2. Average Speed (with trend)
3. Emergency Responses (NEW - with trend)
4. Congestion Level (with trend)

Charts (4)

1. **Traffic Volume Trends** - Line chart with area fill
2. **Congestion Heatmap** - Day x Hour heatmap
3. **Vehicle Distribution** - Donut chart
4. **Emergency Activity** - Stacked bar chart (NEW)

New Sections

1. Emergency Vehicle Activity

- Stacked bar chart
- By type over time

2. Traffic Pattern Analysis

- List of patterns
- Confidence badges
- Recommendations
- Color-coded confidence



Report Templates

1. Comprehensive Report

- All metrics included
- Full analysis
- Emergency data
- Traffic patterns

2. Traffic Volume Summary

- Vehicle counts
- Distribution charts
- Volume trends

3. Congestion Analysis

- Heatmaps
- Peak hours
- Congestion metrics

4. Emergency Response Report (NEW)

- Emergency statistics
- Response types
- Green waves
- Priority analysis

5. Traffic Pattern Analysis (NEW)

- Detected patterns
- Confidence scores
- Recommendations
- TL-specific analysis

6. System Performance

- AI metrics
- TL performance
- Optimization rates

File Generation Features

PDF Reports

- Professional formatting
- Colored headers
- Data tables
- Summary statistics
- Charts data (textual)

Excel Reports

- Multiple worksheets
- Formatted headers
- Conditional formatting
- Summary sheet
- Pattern details

CSV Export

- Raw data
- Quick analysis
- Import to other tools

Data Flow

User → UI Filter → API Request → Database Query →
Data Processing → Response → Chart Update → Display

For Report Generation:

User → Select Metrics → Generate Button →
Backend Collection → File Generation →
Download to Browser

Database Queries

Emergency Analytics Example

```
python

emergency_logs = EmergencyVehicleLog.query.filter(
    EmergencyVehicleLog.scenario == scenario,
    EmergencyVehicleLog.detected_at >= start_date,
    EmergencyVehicleLog.detected_at <= end_date
).all()

# Group by type
by_type = {
    'AMBULANCE': len([l for l in emergency_logs if 'ambulance' in l.vehicle_type.lower()]),
    'FIRE_TRUCK': len([l for l in emergency_logs if 'fire' in l.vehicle_type.lower()]),
    'POLICE': len([l for l in emergency_logs if 'police' in l.vehicle_type.lower()])
}
```

Traffic Pattern Query

```
python

patterns = TrafficPattern.query.filter(
    TrafficPattern.scenario == scenario,
    TrafficPattern.interval_start >= start_date,
    TrafficPattern.interval_end <= end_date
).order_by(TrafficPattern.confidence_score.desc()).all()
```

Key Improvements

Before → After

1. Location Filter → Scenario Filter

- Better alignment with system architecture
- Clearer data separation

2. Static Data → Dynamic Updates

- Real API integration
- Live data refresh

3. Mock Reports → Actual Files

- PDF with ReportLab
- Excel with XlsxWriter
- CSV export

4. Basic Metrics → Comprehensive Analytics

- Emergency tracking
- Pattern analysis
- AI performance

5. Simple UI → Modern Design

- Gradients
- Animations
- Better UX



Installation Checklist

- ☐ Install dependencies: `pip install reportlab XlsxWriter`
- ☐ Copy `report_routes.py` to `app/routes/`
- ☐ Copy `report_service.py` to `app/services/`
- ☐ Replace `report.html` in `app/templates/`
- ☐ Register blueprint in `init.py`
- ☐ Add route to report page
- ☐ Restart Flask server
- ☐ Test all endpoints
- ☐ Generate test report
- ☐ Verify emergency data
- ☐ Check traffic patterns



Testing Commands

```
bash
```

```
# Test traffic overview
```

```
curl "http://localhost:5000/api/reports/traffic-overview?scenario=accra&date_start=2025-06-27&date_end=2025-07-04"
```

```
# Test emergency activity
```

```
curl "http://localhost:5000/api/reports/emergency-activity?scenario=accra&date_start=2025-06-27&date_end=2025-07-04"
```

```
# Test traffic patterns
```

```
curl "http://localhost:5000/api/reports/traffic-patterns?scenario=accra&date_start=2025-06-27&date_end=2025-07-04"
```

```
# Generate PDF report
```

```
curl -X POST http://localhost:5000/api/reports/generate \
```

```
-H "Content-Type: application/json" \
```

```
-d '{
```

```
  "template": "comprehensive",
```

```
  "date_start": "2025-06-27",
```

```
  "date_end": "2025-07-04",
```

```
  "format": "PDF",
```

```
  "metrics": ["Traffic Volume", "Emergency Vehicles", "Traffic Patterns"],
```

```
  "scenario": "accra"
```

```
}' \
```

```
--output report.pdf
```

Common Issues & Solutions

1. No Emergency Data

Problem: Emergency section shows 0

Solutions:

- Check EmergencyVehicleLog table has data
- Verify emergency detection is running
- Check scenario name matches

2. No Traffic Patterns

Problem: Pattern section empty

Solutions:

- Verify TrafficPattern table populated
- Check date range
- Ensure pattern detection service running

Problem: PDF/Excel not downloading

Solutions:

- Install: `pip install reportlab XlsxWriter`
- Check file write permissions
- Review error logs

4. Charts Not Showing

Problem: Empty chart containers

Solutions:

- Check API returns data
- Verify ECharts loaded
- Check console errors

Performance Tips

1. Add Database Indexes

```
sql

CREATE INDEX idx_emergency_scenario_date
ON emergency_vehicle_logs(scenario, detected_at);

CREATE INDEX idx_patterns_scenario
ON traffic_patterns(scenario, confidence_score);
```

2. Implement Caching

```
python

from flask_caching import Cache
cache = Cache(app)

@cache.memoize(timeout=300)
def get_traffic_overview(scenario, start, end):
    # Expensive query
```

3. Limit Date Ranges

```
python
```

```
max_days = 90
if (end_date - start_date).days > max_days:
    return error_response("Date range too large")
```

Success Criteria

Your report system is ready when:

-  All API endpoints return data
-  Emergency chart displays responses
-  Traffic patterns show in list
-  PDF downloads successfully
-  Excel exports work
-  CSV exports function
-  Charts render properly
-  Filters update data
-  No console errors
-  Mobile responsive

Additional Resources

- **ReportLab Docs:** <https://www.reportlab.com/docs/>
- **XlsxWriter Docs:** <https://xlsxwriter.readthedocs.io/>
- **ECharts Docs:** <https://echarts.apache.org/en/index.html>
- **Flask Send File:** https://flask.palletsprojects.com/en/2.3.x/api/#flask.send_file

Future Enhancements

1. Scheduled Reports

- Auto-generate daily/weekly
- Email delivery
- Customizable schedules

2. Advanced Visualizations

- 3D charts
- Geographic heatmaps
- Real-time updates

3. Comparative Analysis

- Compare scenarios
- Year-over-year trends
- Before/after optimization

4. Custom Dashboards

- Drag-and-drop widgets
- Save preferences
- Share reports

5. API Integration

- Webhooks
- External BI tools
- Data streaming

Notes

- All dates use ISO format (YYYY-MM-DD)
- Scenario names are case-sensitive
- Emergency detection must be active
- Pattern detection requires sufficient data
- Report generation has file size limits
- Cache expiration: 5 minutes default

Version: 2.0

Last Updated: November 2025

Status: Production Ready 